Handling Data Heterogeneity and Domain Shift

- **FedProx:** This algorithm adds a proximal term to your local objective function. It restricts local updates to keep local models mathematically close to the global model. You use this to prevent clients with skewed data from ruining your global baseline.

- **SCAFFOLD:** This method uses control variates to correct client drift. It calculates the difference between your local update direction and the global update direction. It then steers your local training back on track to stabilize convergence.

- **FedNova:** This approach normalizes local updates based on the exact number of local steps each client takes. It ensures clients performing more local work do not unfairly dominate your final global model.

- **FedBN:** Federated Batch Normalization tackles non-IID data by keeping batch normalization layers strictly local. You only send the remaining neural network weights to the server. You use this when your clients have severe feature shifts, like different hospitals using completely different MRI scanners.

Personalizing Models for Individual Clients

- **FedPer:** This technique splits your neural network into base layers and personalization layers. You share the base layers globally to learn general features. You keep the final layers completely local to fit your specific user data perfectly.

- **Ditto:** This framework trains a standard global model and a customized local model at the same time. You use a mathematical penalty to keep your local model close to the global one while optimizing for your exact dataset.

Improving Communication Efficiency

- **FedPAQ:** This algorithm quantizes model updates before transmission. It rounds high precision weights down to lower precision formats. You implement this to drastically reduce your network bandwidth costs.

- **FedMA:** Federated Matched Averaging matches hidden elements layer by layer before averaging them. You use this to combine complex neural networks like CNNs more effectively than simple weight averaging.

Enhancing Privacy and Structure

- **DP-Fed:** Differentially Private Federated Learning adds calibrated statistical noise to your local gradients before sharing them. You apply this to mathematically guarantee that bad actors cannot reverse engineer your raw data.

- **FedTree:** This algorithm builds gradient boosting decision trees securely across multiple isolated parties. You deploy this when your organizations share structured tabular data and require strict vertical or horizontal privacy.

Server-Side Optimization

- **FedAdam and FedYogi:** These belong to the FedOpt framework. The server does not just passively average local weights. It treats the aggregated updates as pseudo-gradients and applies adaptive optimizers like Adam or Yogi. You implement this to drastically speed up global convergence.

Machine Unlearning

Machine unlearning is the process of deliberately removing the influence of specific training data points from a trained machine learning model without having to retrain the entire model from scratch.

- When a user requests their data be deleted (often due to privacy regulations like GDPR), or when you discover a dataset contains toxic, biased, or adversarial data, you cannot simply delete the raw file. The neural network has already absorbed the mathematical patterns of that data into its weights.
- The brute-force solution is "Exact Unlearning": you delete the bad data from the dataset, re-initialize the model, and spend days or weeks retraining it. This is computationally expensive and financially ruinous for large foundation models.

- Machine unlearning (often called "Approximate Unlearning") uses mathematical shortcuts to edit the model's weights directly, simulating the effect of the data never having been there.

Class Unlearning: You force the model to forget a massive, global concept. For example, you train a model to completely forget what a "cat" looks like, or you remove all toxic speech from a language model. The footprint is huge, making it easier to erase but harder to do without ruining the model's overall intelligence.

Instance Unlearning: You perform a surgical strike. You want the model to forget 50 specific cat images uploaded by a user who deleted their account, but you want the model to remain perfectly capable of recognizing cats. The footprint is microscopic.

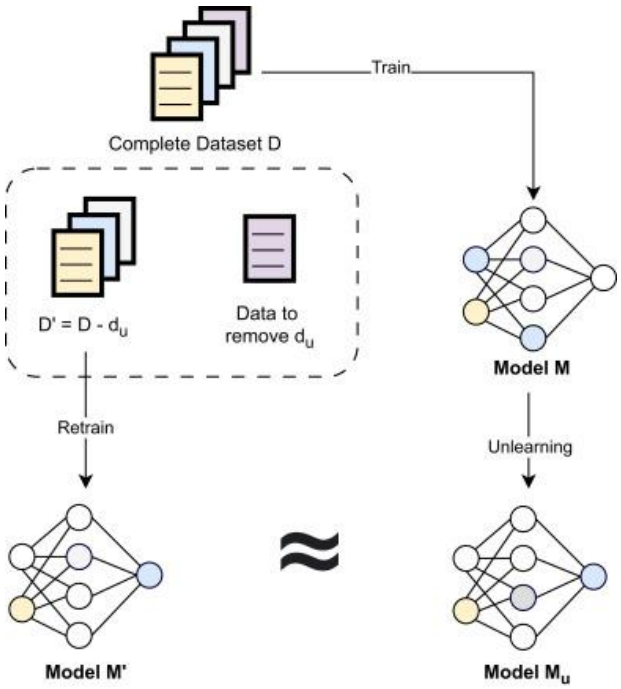
How Machine Unlearning Works

There are several methods, but they generally fall into these categories:

1. Gradient Ascent (The "Amnesia" Method) Normally, when you train a model, you use Gradient Descent to minimize the error on the training data. In unlearning, you do the exact opposite. You isolate the specific data point you want to forget, and you run Gradient *Ascent*. You deliberately update the model's weights to maximize the error on that specific data point, effectively forcing the model to "forget" the pattern. To ensure the model doesn't forget everything else (catastrophic forgetting), this is often paired with a penalty that forces the model to stay close to its original performance on the rest of the good data.

2. Influence Functions (The "Surgical Extraction") This method relies on advanced calculus (Hessian matrices). It mathematically estimates exactly how much a single training image influenced the final weights of the model. Once you calculate this "influence score," you apply an inverse mathematical transformation to subtract that specific influence from the model's parameters, surgically removing the data's footprint.

3. SISA (Sharded, Isolated, Sliced, and Aggregated) This is a structural approach rather than a mathematical one. Before you train, you divide your dataset into completely isolated "shards" and train a separate sub-model on each shard. During inference, you average their predictions. If a user asks to delete their data, you find the exact shard containing their data. You only have to retrain that one small sub-model, leaving the rest of the architecture untouched.


Train
Complete Dataset D
D' = D - d_u
Data to remove d_u
Model M
Retrain
Unlearning
≈
Model M'
Model M_u